# Lab 4.1: M5Core2 IMU Sensors

12 Marcus Frazier

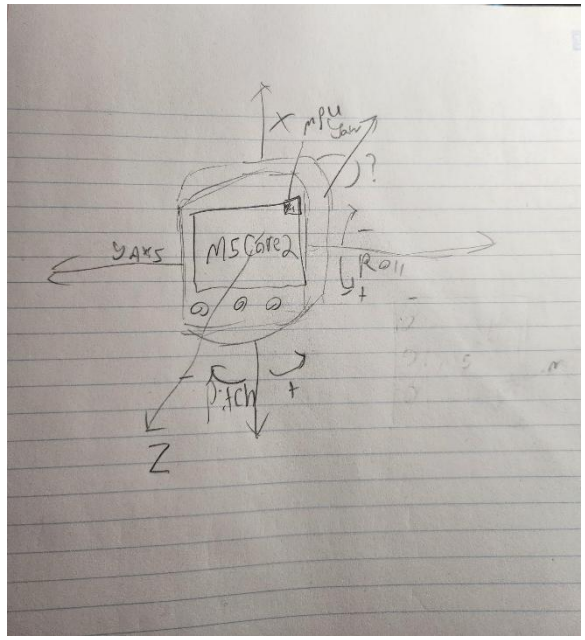
07/26/2025

Professor Prater

Introduction:

The purpose of this lab is to experiment a little with the IMU sensor that is on our M5Cor2's back plate. We can see what it is capable of at the beginning and then we can change it to record max values of data utilizing the accelerometer within, and finally we are going to give it some kind of indicator when certain conditions happen.

Part 1:




This is the sample sketch mpu6886 that gives you data that the MPU is capturing. I found that the MPU chip is positioned about at the top right corner of the screen of our m5Core2's. From there i was able to find that from holding the device from the bottom, the y axis is on the left and right that represents the roll. It goes negative if you roll the device back and positive if you roll it forward. The x axis runs from top to bottom and represents pitch. Spinning right is positive and spinning left is negative. The z axis comes through the front and back of the device where the IMU is located and represents yaw. I wasn't able to indicate the values when you spin it around the axis because it is always changing regardless of its position. I was only able to either speed up or slow down the values. If i held it with the top right corner closest to the ground, the numbers decreased the slowest in that position.

Part 2

Code:

```
*/
#include <M5Core2.h>

float accX = 0.0F;  // Define variables for storing inertial sensor data
float accY = 0.0F;
float accZ = 0.0F;

float gyroX = 0.0F;
float gyroY = 0.0F;
float gyroZ = 0.0F;

float maxX = 0.0F;  // Define variables for storing inertial sensor data
float maxY = 0.0F;
float maxZ = 0.0F;

//macVector value
float maxVec = 0.0F;
float tmpVec = 0.0F;

//to make calcs easier
float xSq = 0.0F;
float ySq = 0.0F;
float zSq = 0.0F;

/* After M5Core2 is started or reset
the program in the setUp () function will be run, and this part will only be run
once. */
void setup() {
    M5.begin();                // Init M5Core.
    M5.IMU.Init();             // Init IMU sensor.
    M5.Lcd.fillScreen(BLACK);  // Set the screen background color to black.
    M5.Lcd.setTextColor(GREEN,BLACK);  // Sets the foreground color and background color of the displayed text.
    M5.Lcd.setTextSize(2);  // Set the font size.
}
```

```
void loop() {
    //add button to clear max data
    //M5.update() check button status
    M5.update();
    // Stores the triaxial gyroscope data of the inertial sensor to the relevant variable
    M5.IMU.getGyroData(&gyroX, &gyroY, &gyroZ);
    M5.IMU.getAccelData(&accX, &accY,&accZ);  // Stores the triaxial accelerometer.

    /* The M5Core screen is 320x240 pixels, starting at the top left corner of
    the screen (0,0). gyroscope output related M5Core2屏幕像素为
    320x240,以屏幕左上角为原点 (0,0)*/
    // gyroscope output related.  陀螺仪输出相关
    M5.Lcd.setCursor(
        0, 20);  // Move the cursor position to (x,y).  移动光标位置到(x,y)处
    M5.Lcd.printf("gyroX,  gyroY, gyroZ");  // Screen printingformatted string.
    M5.Lcd.setCursor(0, 42);
    M5.Lcd.printf("%6.2f %6.2f%6.2f o/s", gyroX, gyroY, gyroZ);

    // Accelerometer output is related
    M5.Lcd.setCursor(0, 70);
    M5.Lcd.printf("accX,   accY,  accZ");
    M5.Lcd.setCursor(0, 92);
    M5.Lcd.printf("%5.2f  %5.2f  %5.2f G", accX, accY, accZ);

    //make sure max is realy maz
    if (accX > maxX){
        maxX = accX;
    }
    if (accY > maxY){
        maxY = accY;
    }
    if (accZ > maxZ){
        maxZ = accZ;
    }
```

```
    // Pose output is related
    M5.Lcd.setCursor(0, 120);
    M5.Lcd.printf("maxX,  maxY,  maxZ");
    M5.Lcd.setCursor(0, 142);
    M5.Lcd.printf("%5.2f  %5.2f  %5.2f deg", maxX, maxY, maxZ);

    // maxVector stuff
    //get X, Y, Z squared
    xSq = sq(accX);
    ySq = sq(accY);
    zSq = sq(accZ);

    tmpVec = sqrt(xSq + ySq + zSq);
    if(tmpVec > maxVec){
        maxVec = tmpVec;
    }

    M5.Lcd.setCursor(0, 175);
    M5.Lcd.printf("MaxVector : %.2f G", maxVec);

    // clear max values from button A
    if (M5.BtnA.wasReleased() || M5.BtnA.pressedFor(1000, 200)) {
        maxX = 0.0F;
        maxY = 0.0F;
        maxZ = 0.0F;
        maxVec = 0.0F;
    }

    delay(100);  // Delay 10ms.
}
```

Output:

```
Flash will be erased from 0x00001000 to 0x00005fff...
Flash will be erased from 0x00008000 to 0x00008fff...
Flash will be erased from 0x0000e000 to 0x0000ffff...
Flash will be erased from 0x00010000 to 0x0007dfff...
Compressed 17568 bytes to 12206...
Writing at 0x00001000... (100 %)
Wrote 17568 bytes (12206 compressed) at 0x00001000 in 0.4 seconds (effective 378.7 kbit/s)...
Hash of data verified.
Compressed 3072 bytes to 146...
Writing at 0x00008000... (100 %)
Wrote 3072 bytes (146 compressed) at 0x00008000 in 0.0 seconds (effective 548.5 kbit/s)...
Hash of data verified.
Compressed 8192 bytes to 47...
Writing at 0x0000e000... (100 %)
Wrote 8192 bytes (47 compressed) at 0x0000e000 in 0.1 seconds (effective 754.5 kbit/s)...
Hash of data verified.
Compressed 446688 bytes to 245148...
Writing at 0x00010000... (6 %)
Writing at 0x00019a9c... (13 %)
Writing at 0x000294fa... (20 %)
Writing at 0x00034967... (26 %)
Writing at 0x0003a042... (33 %)
Writing at 0x0003fb66... (40 %)
Writing at 0x0004517b... (46 %)
Writing at 0x0004acac... (53 %)
Writing at 0x0004fbbe... (60 %)
Writing at 0x00055049... (66 %)
Writing at 0x0005b870... (73 %)
Writing at 0x00064a0d... (80 %)
Writing at 0x0006ce5d... (86 %)
Writing at 0x0007252c... (93 %)
Writing at 0x00077807... (100 %)
Wrote 446688 bytes (245148 compressed) at 0x00010000 in 4.2 seconds (effective 844.9 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

Part 3:

Just added this section for the blinking.

```
    // making the screen flash red and white 5 times when acceleration drops by 10 cm
    if (accZ < 0.1) {
        //Loop so it runs 5 times
        for (int i = 0; i < 5; i++) {
            M5.Lcd.fillScreen(RED);
            delay(250);
            M5.Lcd.fillScreen(WHITE);
            delay(250);
            M5.Lcd.setTextColor(BLACK,WHITE);
        }
        // After blinking go back
        M5.Lcd.fillScreen(BLACK);
        M5.Lcd.setTextColor(GREEN, BLACK);
    }
}
```

Output:

```
Wrote 3072 bytes (146 compressed) at 0x00008000 in 0.0 seconds (effective 543.8 kbit/s)...
Hash of data verified.
Compressed 8192 bytes to 47...
Writing at 0x0000e000... (100 %)
Wrote 8192 bytes (47 compressed) at 0x0000e000 in 0.1 seconds (effective 752.2 kbit/s)...
Hash of data verified.
Compressed 446800 bytes to 245213...
Writing at 0x00010000... (6 %)
Writing at 0x00019ab1... (13 %)
Writing at 0x00029638... (20 %)
Writing at 0x0003497b... (26 %)
Writing at 0x0003a056... (33 %)
Writing at 0x0003fb8d... (40 %)
Writing at 0x000451ad... (46 %)
Writing at 0x0004acc9... (53 %)
Writing at 0x0004fbe1... (60 %)
Writing at 0x0005506d... (66 %)
Writing at 0x0005b8a5... (73 %)
Writing at 0x00064a22... (80 %)
Writing at 0x0006ce7b... (86 %)
Writing at 0x0007254c... (93 %)
Writing at 0x0007782c... (100 %)
Wrote 446800 bytes (245213 compressed) at 0x00010000 in 4.2 seconds (effective 849.4 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

Video link

https://youtube.com/shorts/SV-XXBWZP9M?si=u2z_Zp938BvHYmqB

The code like we did in class is a modified version of the sample code but now we record the max vectors of the X, Y, and Z axis, then it takes all the maxes and adds them together, then takes the square root to get the Max Vector. At the end, we added code to make the screen flash red and white when the accelerometer for z falls by to cm or 0.1. I did try to have all 3 axis using or statements but got constant flashing as I noticed that accX and

accY constantly are reading below 0.1 and triggers the loop. Making it only read accZ, it runs properly.